

Graph-Based Malware Detection Using Opcode Sequences

Sibel Gülmez
Computer Engineering
Gebze Technical University
Kocaeli, Turkey
sgulmez2018@gtu.edu.tr

Ibrahim Sogukpinar
Computer Engineering
Gebze Technical University
Kocaeli, Turkey
ispinar@gtu.edu.tr

Abstract—The impact of malware grows for IT (information technology) systems day by day. The number, the complexity, and the cost of them increase rapidly. While researchers are developing new and better detection algorithms, attackers are also evolving malware to fail the current detection techniques. Therefore malware detection becomes one of the most challenging tasks in cyber security. To increase the performance of the detection techniques, researchers benefit from different approaches. But some of them might cost a lot both in time and hardware resources. This situation puts forward fast and cheap detection methods. In this context, static analysis provides these utilities but it is important to keep detection accuracy high while reducing resource consumption. Opcodes (operational codes) are commonly used in static analysis but sometimes feature extraction from opcodes might be difficult since an opcode sequence might have a great length. Furthermore, most of the malware developers use obfuscation and encryption techniques to avoid detection methods based on static analysis. This kind of malware is called packed malware and according to common belief, packed malware should be either unpacked or analyzed dynamically in order to detect them. In this study, a graph-based malware detection method has been proposed to overcome these problems. The proposed method relies on obtaining the opcode graph of every executable file in the dataset and using them for future extraction. In this way, the proposed method reaches up to 98% detection accuracy. In addition to the accuracy rate, the proposed method makes it possible to detect packed malware without the need for unpacking or dynamic analysis.

Keywords—malware detection, static analysis, opcode analysis, graph-based detection, packed malware

I. INTRODUCTION

The development of the technology provides numerous functionalities but also leads to more and more dangers as well. Among these dangers, malware is the most frequently encountered [1]. Malware is the abbreviation of malicious software. This software is developed by an attacker and might have various harmful goals. Even though there are a great number of detection techniques and tools, malware is evolving to avoid them. PurpleSec statistics show that 80% of the small businesses get infected despite they have antivirus protection [2]. Up-to-dateness of an antivirus software is crucial while 230.000 new malwares are being generated every day [2].

Statistics put forward that the malware detection studies should maintain their continuity.

Various analysis techniques have been used so far in order to detect malware. Signature-based detection is widely used by commercial antivirus softwares due to low time complexity and low false positive rate. But they are vulnerable to repackaging techniques. Studies show that popular signature-based antivirus softwares' detection accuracy rates are dramatically decreasing against repacked malwares [3].

Anomaly-based detection relies on classifying software as benign or malware according to its behaviors. These behaviors can be tracked by using static and/or dynamic analysis. Dynamic analysis requires the executable file to be running, therefore the need for the detection time increases. Static analysis, on the other hand, requires less time to obtain important information. Disassembling a software reveals opcode sequences that are useful for a binary classification between benign and malware.

A graph representation of an executable file is easily obtainable since the opcode sequences have a suitable structure to be transformed into a graph. Representing an executable file as a graph allows implementing graph-based methods like graph compression [4] and graph embedding [5] to achieve distinguishing features of malware and benign files. Analyzing malware from a different perspective provides a possibility to understand the characteristic features of malware. This advantage highlights the effect of using graphs for malware detection.

In addition, one of the most important problems in malware detection is packed malware. A packed malware uses obfuscation and encryption techniques to cover up its behaviors. Therefore, detecting packed malware is becoming another challenge for researchers. Unpacking them or dynamically tracing them requires both more time and resources. Also, not every packed malware is unpackable for now. The optimum solution would be detecting them statically without unpacking.

In this study, a malware detection method based on graphs of opcode sequences has been proposed. This method focuses on the degrees of each node of a sub-graph. Our contributions are detecting malware through a powerful representation without executing them, indicating that it is possible to extract more useful features from opcodes with the help of graphs and

also detecting packed malware without the need of unpacking with a detection rate of 98%.

This paper is organized as follows: Section II presents a literature review of malware detection, Section III describes the proposed method, Section IV presents the experimental results of the proposed method with the experimental setup processes, Section V concludes the paper with possible future work ideas.

II. RELATED WORK

In recent years, malware detection studies increasing both quantitatively and qualitatively to prevent the growing threat of malware. In this section, we highlight some of these works.

Amer et al. [6] propose a detection method based on API (application programming interface) calls by using word embedding in order to understand the relationship between them. They also show that API sub-sequence can be used to predict whether a file is malicious or not. Elhadi et al. [7] implemented a graph matching algorithm on API call graphs. Since graph matching is an NP-complete problem, the authors simplified the data dependent API call graphs in order to reduce the complexity. In that way, they obtained a running time that is less than the running time of LCS (Longest Common Subsequence) even though they used an LCS-based graph matching algorithm. Kim et al. [8] extracted the API calls of malwares to generate the behavioral sequence chains of them. They detect and classify malware by evaluating the similarity of the API call sequence of a target file and the behavioral sequence chains of malware. Ahmadi et al. [9] also proposed a detection method based on dynamic analysis. As distinct from the previous studies, they focused on the repetitive occurrences performed by a software and developed a framework by using iterative patterns of API calls.

Yewale et al. [10] extracted opcode sequences of PE (portable executable) files and used the 20 most frequent opcodes for classification between malware and benign. Four different machine learning algorithms were used to classify the files as either malware or benign. In this way, they obtained an accuracy rate of 97% with the Random Forest classifier. However they used a small dataset with only 100 PE files, therefore the accuracy rate might reduce when the method is performed on a larger dataset. Santos et al. [11] used opcode sequences to obtain a vector representation of executable files. Thus, they developed a detection method that detects unknown malware with low false positives. Zolotukhin et al. [12] also purposed a detection mechanism to detect zero-day malware by using n-grams of opcodes sequences. Zhao et. al [13] used the control-flow construct to keep the long opcode sequences as features. Jeon et al. [14] present a detection method with a convolutional recurrent neural network architecture. Their method relies on compressing long opcode sequences and using them as the input of dynamic recurrent neural network for detecting the malware. Another deep learning based detection method has been proposed by Yuxin et al. [26]. The authors used DBN (deep belief network) which can learn from unlabeled data.

Runwal et al. [25] computed the similarity score of programs by using opcode graphs. Their method focuses on metamorphic malware. Anderson et al. [15] proposed a graph-based algorithm by using the dynamically tracked data. They formed a Markov chain considered as a weighted and directed graph. Kong et al. [16] employed an ensemble of classifiers for malware detection. They used the function call graphs to calculate the similarity between a pair of malware. Park et al. [17] used graph algorithms for the malware classification problem. They classified malwares from different families with a method based on the maximal common sub-graph of the behavioral graphs. Ding et al. [18] used dynamic analysis to obtain the system call dependency graphs. They proposed a detection method based on graph matching. Call graphs also used for malware classification [19], [20] and Android malware detection [21], [22].

III. PROPOSED METHOD

In this study, the executable files are represented as graphs. We hypothesize that the classification of the graphs leads to the detection of malware. For this purpose, various processes have been realized. The first step is to disassemble all files in the dataset. Disassembling exposes the opcodes of a file and these opcodes form an opcode sequence. An opcode sequence contains crucial information about an executable file's actions without running it. Until now, researchers widely benefited from opcode sequences by using different features like sub opcode sequences, opcode frequency, and n-grams. But employing graphs is not that common yet to detect malware.

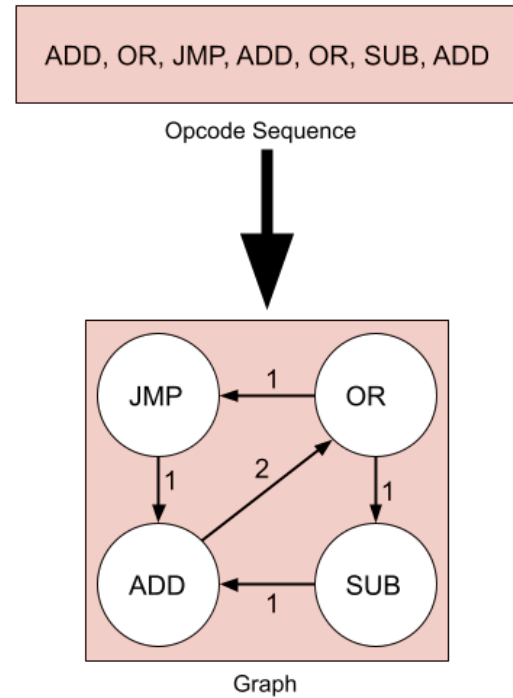


Fig. 1. Visualization of graph generation.

Graphs have a quite useful data form to address the feature extraction process with a different perspective. As seen in Fig. 1 an opcode sequence is easily convertible to a weighted and directed graph with consideration of opcodes as nodes (V) and transition between opcodes as edges (E). Hence, an executable file is represented as $G(V, E)$.

TABLE I
COMPARISON OF THE # OF BUCKLES

Dataset	# of Samples	Overall # of Buckles
Malware	7500	0.000075
Benign	7500	0.000047

Once an opcode sequence of a malware is observed, it can be seen that some opcodes are repeating themselves constantly. The number of the repetitions usually reaches up to thousands. But an opcode histogram, which we consider as the baseline method, only keeps the frequency of the opcodes, the relation between them remains unknown. This led us to analyze buckles in the graphs. According to the test results that are shown in Table I, a malware file has almost 1.6 times more consecutive opcode repetition than a benign file. Therefore, we created sub-graphs by removing edges that connect different opcodes. In this way, a data type which is more useful to differentiate the benign and malware has been obtained. Finally, a histogram file of the sub-graphs, that holds the degrees of each node, has been generated. Fig. 2 represents the generation of a node degree histogram along with the generation of the opcode histogram. The process of node degree histogram creation, described in detail in Algorithm 1.

Algorithm 1 : Creation of Node Degree Histogram

Input: Set of graphs (G), Set of nodes (N)

Output: Node Degree Histogram (H)

$n \leftarrow \text{size of } G$

$m \leftarrow \text{size of } N$

Initialize H as a 2-d array ($n \times m$) with zeros

for each $g_i \in G$ **do**

for each edge (n_1, n_2) of g_i **do**

if $n_1 == n_2$

$f \leftarrow \text{frequency of edge}$

$j \leftarrow \text{index of } n_1 \text{ in } N$

$H_{i,j} = f$

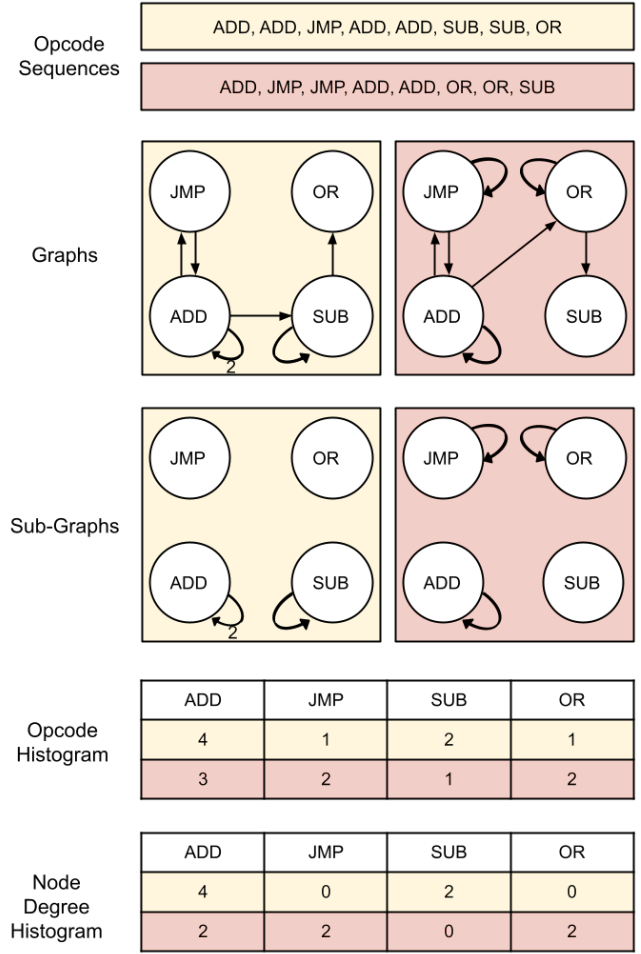


Fig. 2. The difference between the histograms (Two different colors stand for two different PE files)

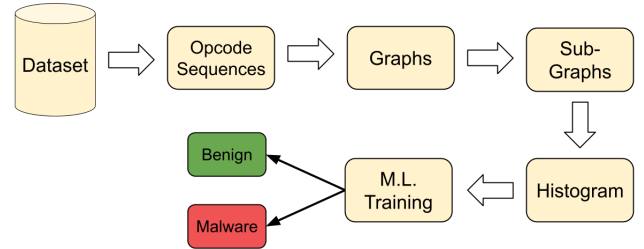


Fig. 3. Framework of the proposed method (M.L: Machine Learning)

All steps of the proposed method are visualized in Fig. 3. These steps are:

- Disassembling the PE file and obtaining the opcode sequence.
- Constructing the graph of the opcode sequence.
- Constructing the sub-graph.
- Creating the histogram file of the node degrees.
- Training and testing the M.L. (Machine Learning) algorithm.
- Labeling the file.

IV. EXPERIMENTAL RESULTS

A. Dataset and Experimental Setup

The dataset we used consists of 7500 malware samples from VX-Heaven and 7500 benign samples from Download.com. Each file, whether benign or malware, is an executable file with the extension of *.exe*. Malwares are from different malware families such as Trojan, Virus, Backdoor, Worm, Rootkit, and Exploit. Also, 5388 out of 7500 malwares were labeled as packed by using PeiD [23]. To evaluate the performance of the proposed method over packed malware, the whole dataset

is considered as the combination of packed and nonpacked files.

Distorm3 disassembler was used to disassemble the files. Python has been utilized as the programming language with useful libraries including machine learning algorithms. All experiments executed on a computer with Core i7 9700K CPU working 3.6GHz processor and 64GB memory running 64bit Ubuntu as the operating system.

B. Evaluation Metrics

Several metrics have been used to evaluate the proposed method. Definitions and formulas are as follows:

- *True positive (TP)*: Number of correctly classified malware.
- *False positive (FP)*: Number of misclassified malware.
- *True negative (TN)*: Number of correctly classified benign.
- *False negative (FN)*: Number of misclassified benign.

$$Recall = \frac{TP}{TP + FN} \quad (1)$$

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

$$TruePositiveRate(TPR) = \frac{TP}{TP + FN} \quad (3)$$

$$FalsePositiveRate(FPR) = \frac{FP}{FP + TN} \quad (4)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

C. Performance Evaluation

In this section, we evaluate the experimental results and compare our method with others in order to emphasize the efficiency of it. First, several tests have been conducted to select the machine learning algorithm that suits best our method. Test results demonstrated in Table II show that our method can achieve a detection accuracy of 98% with the Random Forest classifier. Because of that, in the rest of this paper, Random Forest has been used as the machine learning algorithm for all experiments.

TABLE II
PERFORMANCE COMPARISON TO DIFFERENT CLASSIFIERS

Classifier	Precision	Recall	Accuracy
Random Forest	0.98	0.96	0.98
Decision Tree	0.97	0.96	0.97
SVM	0.88	0.94	0.92
KNN (k=1)	0.82	0.82	0.82
KNN (k=2)	0.80	0.80	0.80
KNN (k=3)	0.81	0.80	0.80
KNN (k=4)	0.82	0.81	0.81
KNN (k=5)	0.82	0.80	0.86

The tests whose results are shown in Table II and also in Table IV were conducted on all datasets. But as mentioned

in Section III, our malware dataset is considered as follows: packed malware, nonpacked malware, and all of them. Here-with, these 3 datasets have been tested separately. Table III displays the results of these tests. According to these results, it can be inferred that it is possible to detect packed malwares without unpacking them with a good representation of their features that collected with static analysis. Even though the packing process aims at overcoming the source code analysis, the packer leaves a mark that can be traced through static analysis.

TABLE III
PERFORMANCE COMPARISON TO THE OPCODE HISTOGRAM WITH DIFFERENT DATASETS

Method	Dataset	Precision	Recall	Accuracy
Proposed Method	Packed (P)	0.95	0.97	0.97
	Nonpacked (N)	0.99	0.97	0.97
	P+N	0.98	0.96	0.98
Opcode Histogram	Packed (P)	0.94	0.85	0.90
	Nonpacked (N)	0.91	0.97	0.90
	P+N	0.81	0.95	0.89

Table III also shows the proposed method's performance when it's compared to the opcode histogram. An opcode histogram of all executable files has been generated to see the difference between the performances. The test results show that the node degree histogram keeps more valuable data than the opcode histogram. As can be seen, there is a difference of up to 9% in terms of detection accuracy.

Another important conclusion that can be drawn from Table III is that even the opcode histogram is able to detect packed malware with the same accuracy rate of nonpacked files as in the proposed method. Both in opcode histogram and the proposed method, the accuracy rate only differs when the packed and nonpacked malware is combined. It can be inferred that whether packed or nonpacked, the malware can be easily detected through static analysis. But if they are mixed in a dataset, the accuracy rate might either increase or decrease according to the features which have been utilized in the detection method that used, since it might seem like a multi-classification of packed malware, nonpacked malware, and benign.

TABLE IV
PERFORMANCE COMPARISON TO OTHER METHODS (— STANDS FOR THE UNSHARED INFORMATION)

Method	Dataset	TPR	FPR	Accuracy
Opcode Vectorization [11]	VXHeaven	0.94	0.02	0.96
Opcode Frequency [10]	—	0.91	0.00	0.97
Opcode Frequency [24]	VXHeaven	—	—	0.94
Opcode Graph Embedding [5]	VXHeaven	—	—	0.96
Proposed Method	VXHeaven	0.96	0.02	0.98

The proposed method is also compared to other methods in the literature for the purpose of an efficient evaluation. For a fair comparison as much as possible, we investigated the

studies which are similar in terms of method and dataset to the proposed method. The comparison is shown in Table IV. As seen in the table, the proposed method outperforms similar detection techniques with regard to accuracy and TPR. The source of the dataset [10] used is unknown but that dataset consists of only 100 files. This indicates that the FPR might significantly increase with a larger dataset.

V. CONCLUSION

In this paper, we represented PE files as graphs to benefit from the graph operations. In this way, more meaningful features have been extracted from an opcode sequence obtained by disassembling. To the best of our knowledge, we proposed a novel detection method by using the histogram of node degrees which is considered a successful method with a detection accuracy of 98%.

Another remarkable contribution of our method is that it can also overcome the packed malware which is one of the most challenging problems of malware detection. Despite the attackers' effort to hide the harmful parts of the code, the packing process forms a pattern among packed malwares. Thus, with an effective representation, it becomes possible to detect them without the need for unpacking.

To extend the proposed method, one of the future works will be implementing node selection which might lead to more effective feature extraction. Also, some of the important features might be lost due to the edge removing process. This process might limit the proposed method's detection rate. Therefore another selection method over the edges, which will be deleted, may help to overcome this loss.

The proposed method is implemented only one dataset which is mentioned in Section IV. For this reason, implementation on different datasets of the proposed method will be one of the future works to determine the data dependency of the method. Also, investigating the packed malwares in the dataset we used, might reveal new information about the detection of packed malware. This may lead to performance improvement for the detection of brand new packers that haven't been encountered before.

REFERENCES

- [1] McAfee Labs Threats Reports Nov 2020. Accessed February, 2021. <https://www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-nov-2020.pdf>
- [2] PurpleSec - 2020 Cyber Security Statistics. Accessed February, 2021. <https://purplesec.us/resources/cyber-security-statistics/>
- [3] Yuan, Z., Lu, Y., & Xue, Y. (2016). Droiddetector: android malware characterization and detection using deep learning. *Tsinghua Science and Technology*, 21(1), 114-123.
- [4] Wüchner, T., Cislak, A., Ochoa, M., & Pretschner, A. (2017). Leveraging compression-based graph mining for behavior-based malware detection. *IEEE Transactions on Dependable and Secure Computing*, 16(1), 99-112.
- [5] Hashemi, H., Azmoodeh, A., Hamzeh, A., & Hashemi, S. (2017). Graph embedding as a new approach for unknown malware detection. *Journal of Computer Virology and Hacking Techniques*, 13(3), 153-166.
- [6] Amer, E., & Zelinka, I. (2020). A dynamic Windows malware detection and prediction method based on contextual understanding of API call sequence. *Computers & Security*, 92, 101760.
- [7] Elhadi, A. A. E., Maarof, M. A., & Barry, B. I. (2013). Improving the detection of malware behaviour using simplified data dependent API call graph. *International Journal of Security and Its Applications*, 7(5), 29-42.
- [8] Kim, H., Kim, J., Kim, Y., Kim, I., Kim, K. J., & Kim, H. (2019). Improvement of malware detection and classification using API call sequence alignment and visualization. *Cluster Computing*, 22(1), 921-929.
- [9] Ahmadi, M., Sami, A., Rahimi, H., & Yadegari, B. (2013). Malware detection by behavioural sequential patterns. *Computer Fraud & Security*, 2013(8), 11-19.
- [10] Yewale, A., & Singh, M. (2016, May). Malware detection based on opcode frequency. In *2016 International Conference on Advanced Communication Control and Computing Technologies (ICACCT)* (pp. 646-649). IEEE.
- [11] Santos, I., Brezo, F., Ugarte-Pedrero, X., & Bringas, P. G. (2013). Opcode sequences as representation of executables for data-mining-based unknown malware detection. *Information Sciences*, 231, 64-82.
- [12] Zolotukhin, M., & Hämmäläinen, T. (2014, January). Detection of zero-day malware based on the analysis of opcode sequences. In *2014 IEEE 11th Consumer Communications and Networking Conference (CCNC)* (pp. 386-391). IEEE.
- [13] Zhao, Z., Wang, J., & Bai, J. (2014). Malware detection method based on the control-flow construct feature of software. *IET Information Security*, 8(1), 18-24.
- [14] Jeon, S., & Moon, J. (2020). Malware-detection method with a convolutional recurrent neural network using opcode sequences. *Information Sciences*, 535, 1-15.
- [15] Anderson, B., Quist, D., Neil, J., Storlie, C., & Lane, T. (2011). Graph-based malware detection using dynamic analysis. *Journal in computer virology*, 7(4), 247-258.
- [16] Kong, D., & Yan, G. (2013, August). Discriminant malware distance learning on structural information for automated malware classification. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining* (pp. 1357-1365).
- [17] Park, Y., Reeves, D., Mulukutla, V., & Sundaravel, B. (2010, April). Fast malware classification by automated behavioral graph matching. In *Proceedings of the Sixth Annual Workshop on Cyber Security and Information Intelligence Research* (pp. 1-4).
- [18] Ding, Y., Xia, X., Chen, S., & Li, Y. (2018). A malware detection method based on family behavior graph. *Computers & Security*, 73, 73-86.
- [19] Kinable, J., & Kostakis, O. (2011). Malware classification based on call graph clustering. *Journal in computer virology*, 7(4), 233-245.
- [20] Park, Y., Reeves, D., Mulukutla, V., & Sundaravel, B. (2010, April). Fast malware classification by automated behavioral graph matching. In *Proceedings of the Sixth Annual Workshop on Cyber Security and Information Intelligence Research* (pp. 1-4).
- [21] Pektaş, A., & Acarman, T. (2020). Deep learning for effective Android malware detection using API call graph embeddings. *Soft Computing*, 24(2), 1027-1043.
- [22] Gascon, H., Yamaguchi, F., Arp, D., & Rieck, K. (2013, November). Structural detection of android malware using embedded call graphs. In *Proceedings of the 2013 ACM workshop on Artificial intelligence and security* (pp. 45-54).
- [23] Peid. Accessed March, 2021. <https://www.aldeid.com/wiki/PEiD.pdf>
- [24] Woo, S. U., Kim, D. H., & Chung, T. M. (2016). Method of detecting malware through analysis of opcodes frequency with machine learning technique. In *Advances in Computer Science and Ubiquitous Computing* (pp. 1019-1024). Springer, Singapore.
- [25] Runwal, N., Low, R. M., & Stamp, M. (2012). Opcode graph similarity and metamorphic detection. *Journal in computer virology*, 8(1), 37-52.
- [26] Yuxin, D., & Siyi, Z. (2019). Malware detection based on deep learning algorithm. *Neural Computing and Applications*, 31(2), 461-472.